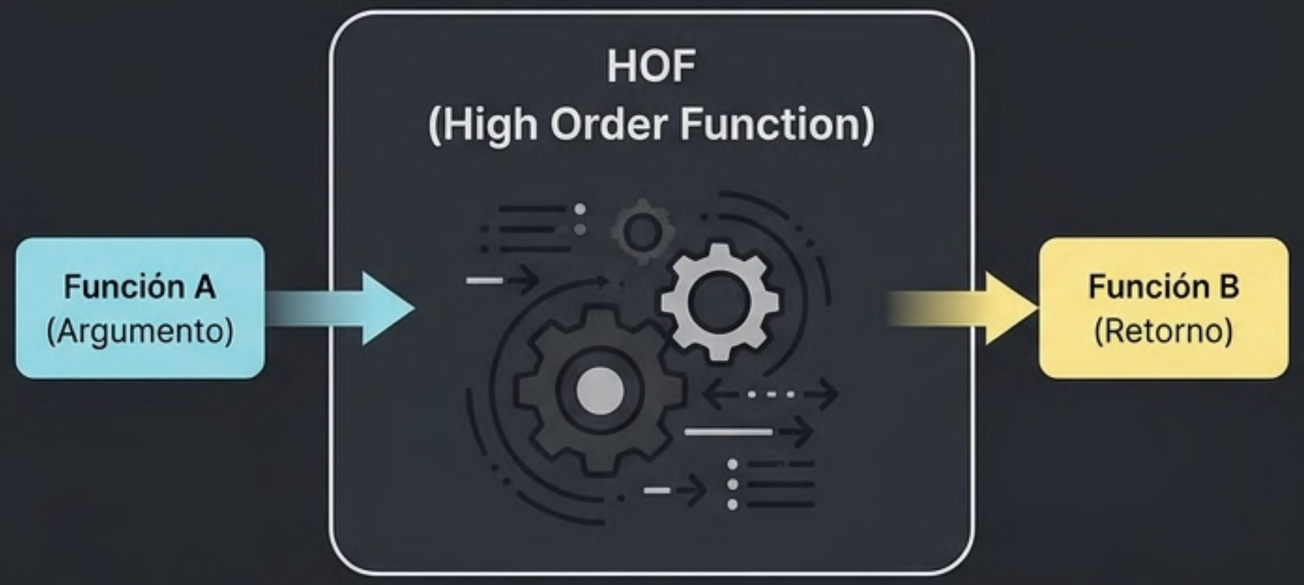
# Dominando las Funciones de Orden Superior

De la iteración básica a la composición de datos avanzada en **JavaScript.**

```
const data = [1, 2, 3];
const processed = data.map(x => x * 2).reduce((a, b) => a + b, 0);

function createProcessor(fn) {
  return (arr) => arr.filter(fn);
}

// Functional Composition
const arr =
  reduce((arr) => {
    return  {
      map(a, b) => 1,
      .map[(a * e) =>
      return 6+ + a => 2;
    },
  ]);
}

function createProcessor(fn) {
  return (arr) => arr.filter(fn);
}

// Functional Composition Example
const fn, => fn,} = arr.fn;
```

NotebookLM

# El Fundamento: ¿Qué es una Función de Orden Superior?




- **Ciudadanos de 1ra Clase:** Las funciones se tratan como valores regulares.
- **Comportamiento:** Una HOF recibe funciones como argumentos o las retorna.
- **Propósito:** Ocultan la complejidad de implementación (cómo) para enfocarnos en la intención (qué).

```
// HOF que abstrae un bucle 'for'
function repeat(n, action) {
  for (let i = 0; i < n; i++) {
    action(i);
  }
}

// Pasamos console.log como la 'acción'
repeat(3, console.log);
// Salida: 0, 1, 2
```

NotebookLM

# El Habilitador: Arrow Functions (=>)

Las HOFs requieren pasar callbacks constantemente. Las **Arrow Functions** ofrecen una sintaxis ultra-concisa que reduce drásticamente el ruido visual, eliminando llaves, retornos explícitos y paréntesis redundantes.

**Antes** (Function Expression)

```
let double = function(n) {
  return n * 2;
};
```

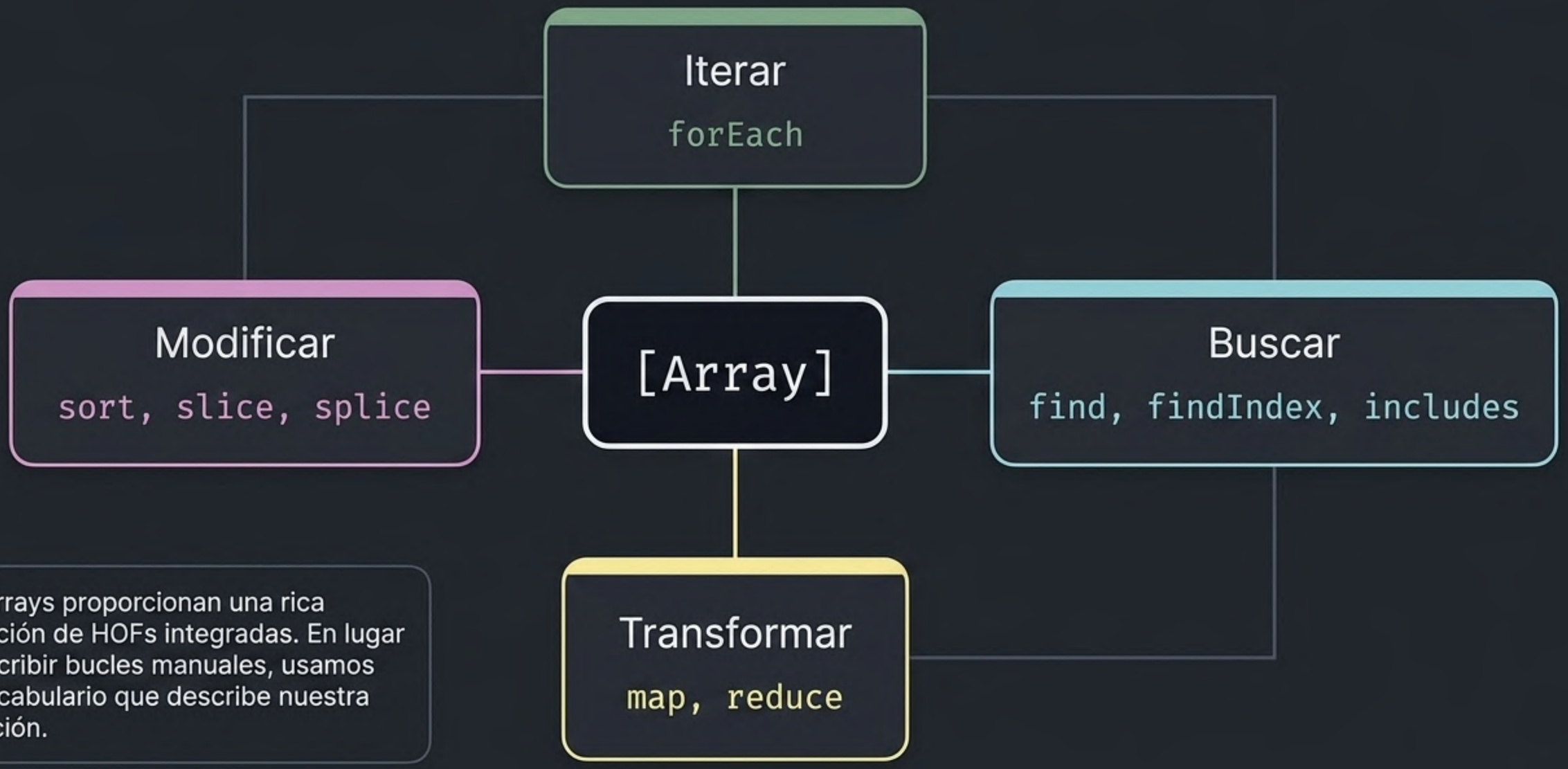
**Ahora** (Arrow Function)

```
let double = n => n * 2;
```

- ✅ Retorno implícito sin llaves { }
- ✅ Sin paréntesis para un solo argumento
- ✅ Código declarativo en una sola línea

NotebookLM

# La Navaja Suiza de los Datos: Métodos de Array




Los arrays proporcionan una rica colección de HOFs integradas. En lugar de escribir bucles manuales, usamos un vocabulario que describe nuestra intención.

NotebookLM

# Iterando con Propósito: `forEach`

```
const heroes = ['Bilbo', 'Gandalf', 'Nazgul'];
for (let i = 0; i < heroes.length; i++) {
  console.log(`${heroes[i]} está en el índice ${i}`);
}
```

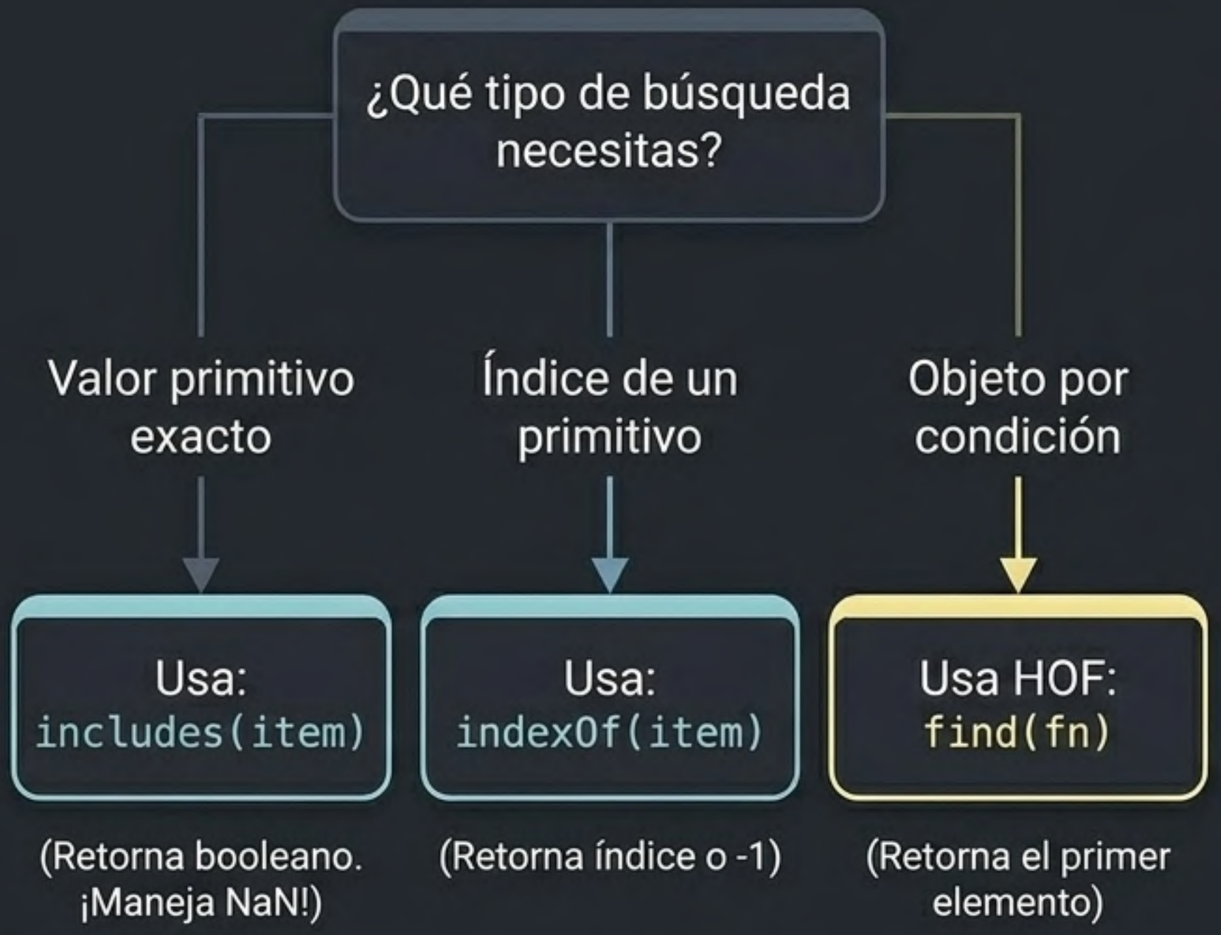
Anti-Pattern (Iteración Manual)

```
const heroes = ['Bilbo', 'Gandalf', 'Nazgul'];

heroes.forEach((item, index) => {
  console.log(`${item} está en el índice ${index}`);
});

// Salida:
// Bilbo está en el índice 0
// Gandalf está en el índice 1...
```

Ejecuta una función por cada elemento puro. Se usa exclusivamente para generar efectos secundarios (ej. imprimir o modificar DOM), ya que su valor de retorno es siempre ignorado.

NotebookLM

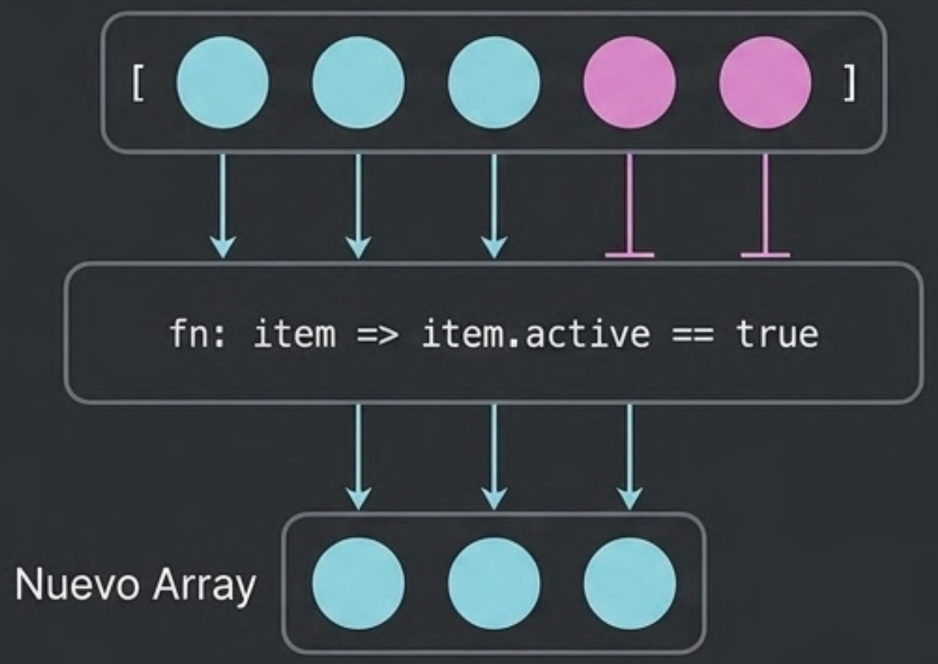
# Encontrando Datos: `find` vs `includes`




```
let users = [
  { id: 1, name: 'John' },
  { id: 2, name: 'Pete' }
];

// HOF en acción: Buscando objeto
por propiedad
let user = users.find(item =>
item.id == 1);

console.log(user.name); // 'John'
```

NotebookLM

# Filtrando el Ruido: `filter`

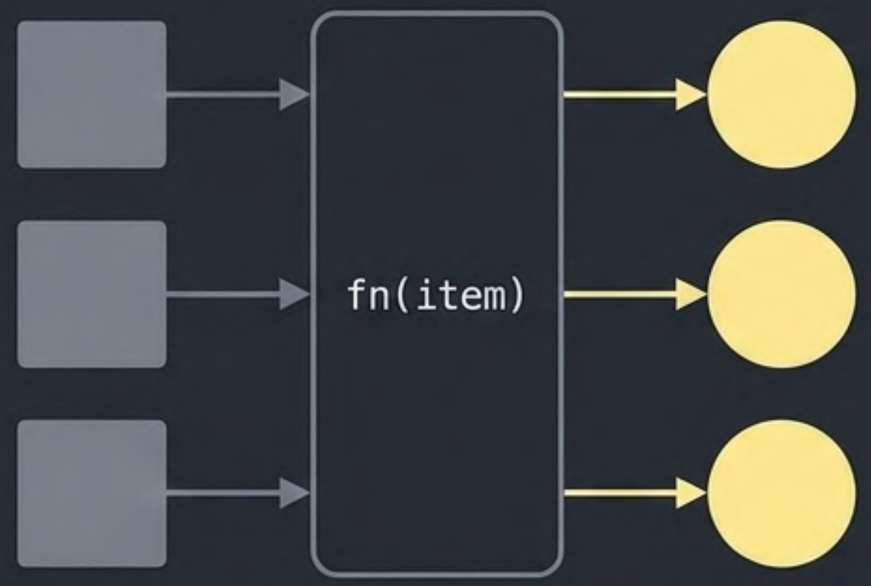



Es una función pura: evalúa todos los elementos y devuelve una copia nueva sin mutar el array original.

```
let scripts = [
  { name: 'Latin', active: true },
  { name: 'Coptic', active: false },
  { name: 'Arabic', active: true }
];

// Extrae solo los scripts activos en un nuevo array
let activeScripts = scripts.filter(s => s.active);

console.log(activeScripts.length); // 2
```

NotebookLM

# Mapeando Nuevas Realidades: map




Transformación 1-a-1: Llama a una función para cada elemento y construye un nuevo array a partir de los resultados devueltos, manteniendo exactamente la misma longitud.

```
1   let users = [
2     { name: 'John', surname: 'Smith', id: 1 },
3     { name: 'Pete', surname: 'Hunt', id: 2 }
4   ];
5
6   // Transformando un array de objetos a array
    de strings
7   let fullNames = users.map(user =>
8       `${user.name} ${user.surname}`
9   );
10
11  console.log(fullNames);
12  // ['John Smith', 'Pete Hunt']
```

NotebookLM

# Reduciendo a la Esencia: reduce

El Flujo de Cálculo para [1, 2, 3, 4, 5]

| Acumulador | Actual | Resultado |
|---|---|---|
| 0 | 1 | 1 |
| 1 | 2 | 3 |
| 3 | 3 | 6 |
| 6 | 4 | 10 |
| 10 | 5 | 15 |

Reduce construye un único valor iterando y arrastrando el resultado anterior.

```
let numbers = [1, 2, 3, 4, 5];


// sum = acumulador, current = actual
// 0 = valor inicial
let sum = numbers.reduce((sum, current) => {
  return sum + current;
}, 0);


console.log(sum); // 15
```

NotebookLM

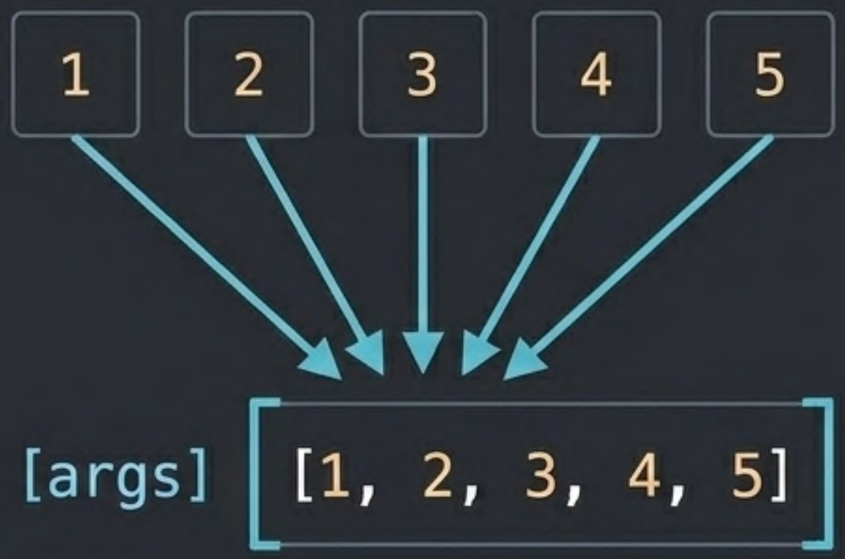
# Cuidado con la Mutación: sort, slice y splice

| Método | Estado | Descripción |
|---|---|---|
| `slice(start, end)` | | Seguro. Copia una porción del array en uno nuevo inmutable. |
| `splice(start, count, ...items)` | | Destructivo. Inserta, remueve y reemplaza elementos en el array original. |
| `sort(fn)` | | Destructivo. Ordena el array original in-place. Convierte a strings por defecto. |

```
1   let arr = [1, 15, 2];
2
3   // ⚠️ TRAMPA: Por defecto convierte a strings ('15' < '2')
4   arr.sort();
5   console.log(arr); // [1, 15, 2]
6
7   // ✅ SOLUCIÓN: Proveer una HOF de comparación
8   arr.sort((a, b) => a - b);
9   console.log(arr); // [1, 2, 15]
```

NotebookLM

# Capturando el Resto: Parámetros Rest (...)

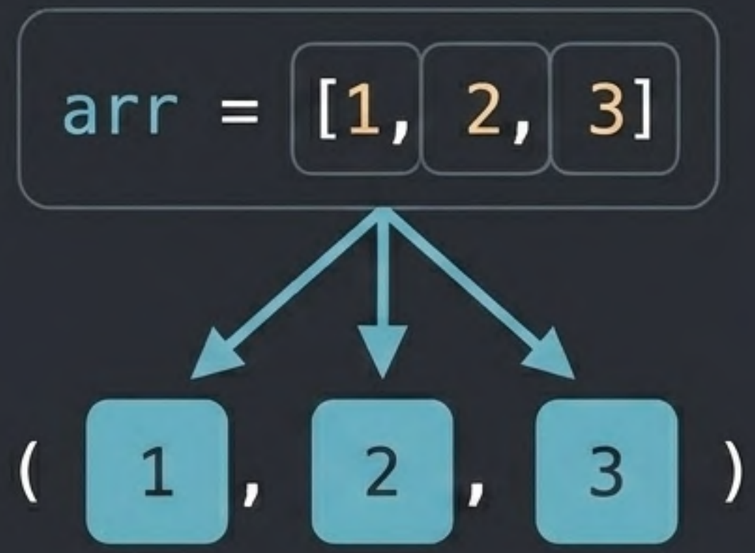



Los tres puntos (...) al final de la declaración de parámetros empaquetan todos los argumentos sobrantes en un array real. Esto nos permite crear funciones flexibles que aceptan un número infinito de argumentos y aplicarles HOFs como reduce.

```
1   // ...args recolecta todos los parámetros
2   function sumAll(...args) {
3     // 'args' es un array real, podemos usar
      reduce
4     return args.reduce((sum, current) =>
5       sum + current, 0
6     );
7   }
8
9   console.log( sumAll(1, 2) ); // 3
10  console.log( sumAll(1, 2, 3, 4, 5) ); // 15
```

NotebookLM

# Desempaquetando Datos: Sintaxis Spread (...)



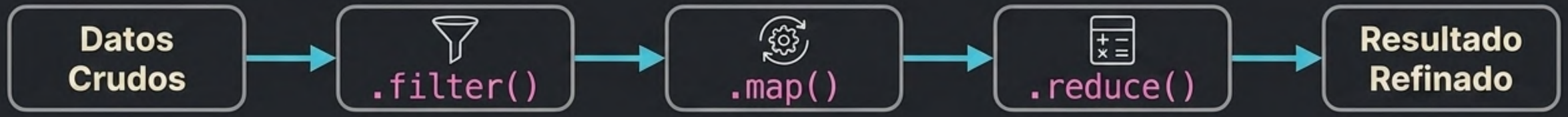

Mientras Rest agrupa, Spread expande. Se usa en la llamada a funciones para desempaquetar iterables en elementos individuales. Es la técnica moderna por excelencia para clonar y fusionar estructuras de datos de forma inmutable.

```
1   let arr1 = [1, -2, 3];
2   let arr2 = [8, 3, -8];
3
4
5   // Expandiendo arrays como argumentos
    separados
6   console.log( Math.max(...arr1, ...arr2) );
7   // 8
8
9   // Clonando y fusionando limpiamente
10  let mergedCopy = [0, ...arr1, ...arr2];
11  console.log(mergedCopy);
12  // [0, 1, -2, 3, 8, 3, -8]
```

NotebookLM

# El Clímax: Composición de Datos (Chaining)

La pureza de `map` y `filter` (retornar nuevos arrays sin mutar) nos permite encadenar métodos en pipelines expresivos.




```javascript
let scripts = [{name: 'Latin', year: 500}, {name: 'Han', year: -1000}];

// Pipeline: Promedio de años post-cristo
let avgYear = scripts
    .filter(s => s.year > 0) // 1. Filtrar > 0
    .map(s => s.year) // 2. Extraer años
    .reduce((sum, yr, i, arr) => // 3. Promediar
      sum + yr / arr.length, 0);

console.log(avgYear); // 500
```

NotebookLM

# Cheat Sheet: Tu Arsenal de Arrays

| Intención | Métodos (HOFs) | ¿Muta el Original? | Retorna |
|---|---|---|---|
| Buscar | `find`, `findIndex` | ❌ No | Elemento o Índice |
| Validar | `includes`, `some`, `every` | ❌ No | Booleano |
| Filtrar | `filter` | ❌ No | Nuevo Array |
| Transformar | `map` | ❌ No | Nuevo Array |
| Agregar | `reduce` | ❌ No | Valor Único |
| Modificar | `sort`, `splice`, `reverse` | ⚠️ SÍ | Array Mutado |
| Extraer | `slice` | ❌ No | Nuevo Array |

*Nota: Siempre favorece operaciones inmutables para construir arquitecturas de datos predecibles y libres de side-effects.*

NotebookLM

# Código Declarativo = Código Elegante

- Oculta la complejidad de la máquina (bucles, contadores).
- Previene bugs y side-effects mediante la inmutabilidad.
- Haz que tu código hable el lenguaje del problema.

```
// EOF. Think in data flows. Keep coding.
```

NotebookLM